

Lecture Unit 4.3

Collective communication

Collective communication

- Using only point-to-point communication would be inconvenient for many operations, particularly with a large numbers of processes
- MPI also has operations for communicating within a group of processes specified by a communicator
- These are referred to as “collective operations” or “collectives”
- Collectives make MPI programming easier and can be more efficient than constructing the corresponding communication from point-to-point calls

MPI collectives

- In this unit we will see examples of four types of MPI collective functions:
 - MPI_Bcast
 - MPI_Scatter
 - MPI_Gather/MPI_Allgather
 - MPI_Reduce/MPI_Allreduce

Broadcast

- Send data from one process to all the others in the specified communicator (one-to-all)
- All processes receive the same data



MPI_Bcast

```
int MPI_Bcast(void *buffer, int count,  
MPI_Datatype datatype, int root,  
MPI_Comm comm)
```

`buffer`: send buffer

`count`: number of items to send

`datatype`: type of data

`root`: rank of sending process

`MPI_Comm comm`: communicator

MPI_Bcast

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]){

    int myRank;
    float number;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);

    if ( myRank == 0 ){
        number=3.141;
    }
    else{
        number=0.0;
    }

    printf("Before bcast: I am process %d The number is %f\n", myRank, number);
    MPI_Bcast(&number,1,MPI_FLOAT,0,MPI_COMM_WORLD);
    printf("After bcast: I am process %d The number is %f\n", myRank, number);

    MPI_Finalize();

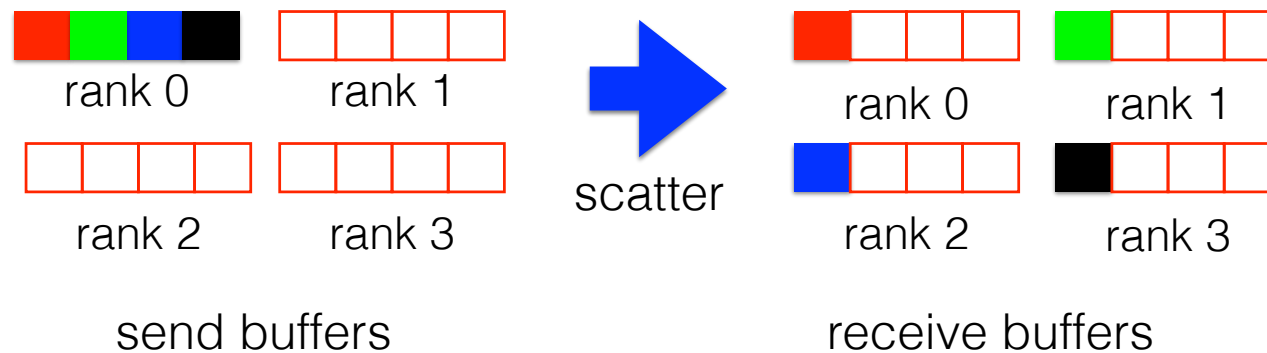
    return 0;
}
```

MPI_Bcast

```
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o bcast bcast.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ ls
bcast  bcast.c  gather.c  gather_malloc.c  hello_mpi  hello_mpi.c  pointtopoint  point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 8 bcast
Before bcast: I am process 4 The number is 0.000000
Before bcast: I am process 5 The number is 0.000000
Before bcast: I am process 6 The number is 0.000000
Before bcast: I am process 7 The number is 0.000000
Before bcast: I am process 1 The number is 0.000000
Before bcast: I am process 2 The number is 0.000000
Before bcast: I am process 0 The number is 3.141000
After bcast: I am process 0 The number is 3.141000
After bcast: I am process 1 The number is 3.141000
Before bcast: I am process 3 The number is 0.000000
After bcast: I am process 3 The number is 3.141000
After bcast: I am process 2 The number is 3.141000
After bcast: I am process 4 The number is 3.141000
After bcast: I am process 5 The number is 3.141000
After bcast: I am process 6 The number is 3.141000
After bcast: I am process 7 The number is 3.141000
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ |
```

Scatter

- Also distributes data from one process to all the others in a communicator (one-to-all)
- With a scatter each process receives a sub-set of the data



MPI_Scatter

```
int MPI_Scatter(const void *sendbuf, int  
sendcount, MPI_Datatype sendtype, void  
*recvbuf, int recvcount, MPI_Datatype  
recvtype, int root, MPI_Comm comm)
```

sendbuf: send buffer

sendcount: number of items to send

sendtype: type of data

recvbuf: receive buffer

recvcount: number of items to receive

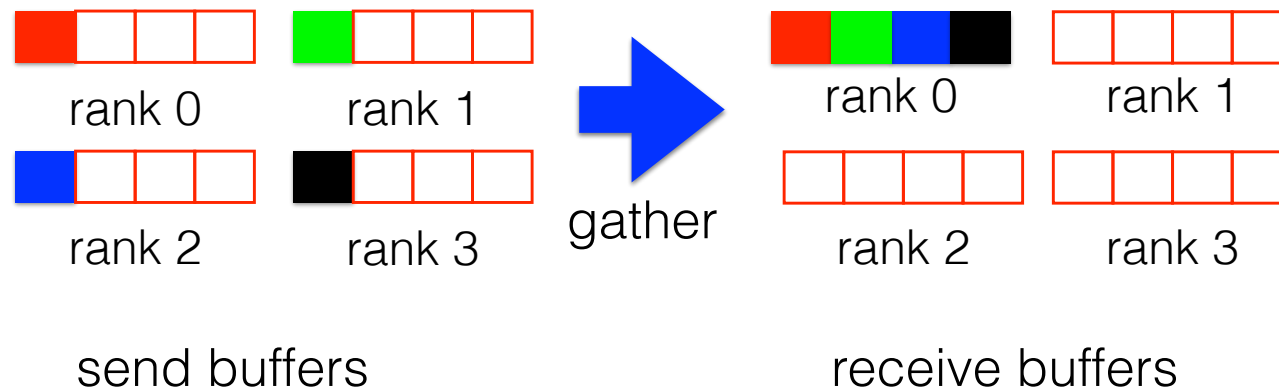
recvtype: type of data

root: rank of process to scatter from

comm: communicator

Gather

- Gather data from all processes in a communicator to one process (all-to-one)
- Each process contributes a sub-set of the data received



MPI_Gather

```
int MPI_Gather(const void *sendbuf, int  
sendcount, MPI_Datatype sendtype, void  
*recvbuf, int recvcount, MPI_Datatype  
recvtype, int root, MPI_Comm comm)
```

sendbuf: send buffer

sendcount: number of elements to send

sendtype: type of data

recvbuf: receive buffer

recvcount: number of elements to receive

recvtype: type of data

root: rank of process to gather to

comm: communicator

MPI_Gather

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]){

    int myRank;                /* Rank of this processes */
    int numProcs;              /* Number of processes */
    const int NUM_VALS=2;      /* Number of values per process */
    int num_local[NUM_VALS];   /* Array to hold values local to a process*/
    const int MAX_PROCS = 10;  /* Maximum number of MPI processes we can handle */
    int num_global[NUM_VALS*MAX_PROCS]; /* Array to hold gathered values*/
    const int root=0;          /* Process to gather to */
    int i;                     /* Loop index */

    /* Set up MPI and get rank and number of processes*/
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
    MPI_Comm_size(MPI_COMM_WORLD, &numProcs);

    /* Store process rank and rank + 10 num_local*/
    num_local[0] = myRank;
    num_local[1] = myRank + 10;

    /* Gather values from all processes onto rank 0 process */
    MPI_Gather(num_local, NUM_VALS, MPI_INT, num_global, NUM_VALS, MPI_INT, root, MPI_COMM_WORLD);

    /* Write out results from the root process */
    if (myRank == root){
        printf("Numbers: ");
        for (i=0; i< NUM_VALS*numProcs; i++){
            printf("%d ", num_global[i]);
        }
        printf("\n");
    }

    /* Close down MPI */
    MPI_Finalize();

    return 0;
}
```

MPI_Gather

```
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ ls
bcast bcast.c gather.c gather_malloc.c hello_mpi hello_mpi.c pointtopoint point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpicc -o gather gather.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ ls
bcast bcast.c gather gather.c gather_malloc.c hello_mpi hello_mpi.c pointtopoint point-to-point.c
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 2 ./gather
Numbers: 0 10 1 11
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 4 ./gather
Numbers: 0 10 1 11 2 12 3 13
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 8 ./gather
Numbers: 0 10 1 11 2 12 3 13 4 14 5 15 6 16 7 17
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ mpirun -np 10 ./gather
Numbers: 0 10 1 11 2 12 3 13 4 14 5 15 6 16 7 17 8 18 9 19
(OSS 2407.0) [jp1143@login02(exeter) TeachingW4]$ |
```

It works but only up to 10 MPI processes.
We can fix this by allocating the memory
required with a call to `malloc` ...

MPI_Gather with malloc

```
#include <stdlib.h>
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]){

    int myRank;          /* Rank of this processes */
    int numProcs;        /* Number of processes */
    const int NUM_VALS=2; /* Number of values per process */
    int num_local[NUM_VALS]; /* Array to hold values local to a process*/
    int *num_global;      /* Pointer to array to hold gathered values*/
    const int root=0;     /* Process to gather to */
    int i;               /* Loop index */

    /* Set up MPI and get rank and number of processes*/
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &myRank);
    MPI_Comm_size(MPI_COMM_WORLD, &numProcs);

    /* Allocate buffer to hold gathered values */
    if (myRank == root){num_global = (int *) malloc(NUM_VALS*numProcs*sizeof(int));}

    /* Store process rank and rank + 10 num_local*/
    num_local[0] = myRank;
    num_local[1] = myRank + 10;

    /* Gather values from all processes onto rank 0 process */
    MPI_Gather(num_local, NUM_VALS, MPI_INT, num_global, NUM_VALS, MPI_INT, root, MPI_COMM_WORLD);

    /* Write out results from the root process */
    if (myRank == root){
        printf("Numbers: ");
        for (i=0; i< NUM_VALS*numProcs; i++){
            printf("%d ", num_global[i]);
        }
        printf("\n");
    }

    /* Free allocated memory */
    if (myRank ==root){free(num_global);}

    /* Close down MPI */
    MPI_Finalize();

    return 0;
}
```

Include `stdlib.h` header file

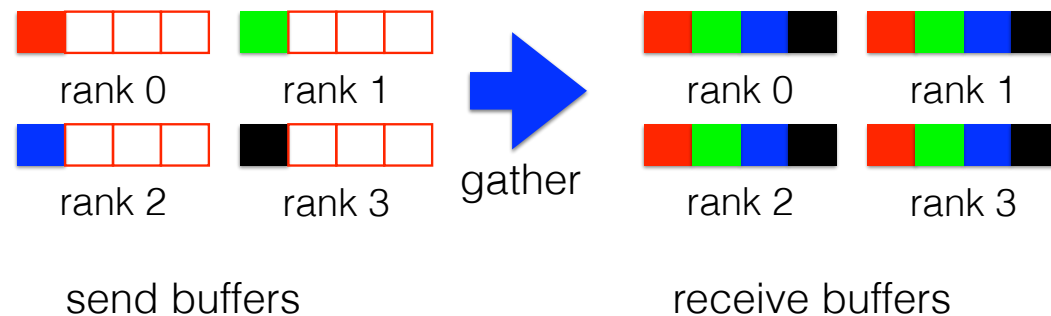
Declare `num_global` as a pointer

Allocate memory on root process

Free memory on root process

Allgather

- Gather data from all processes in a communicator to all processes (all-to-all)
- Each process contributes a sub-set of the data received
- All processes receive the result



MPI_Allgather

```
int MPI_Gather(const void *sendbuf, int  
sendcount, MPI_Datatype sendtype, void  
*recvbuf, int recvcount, MPI_Datatype  
recvtype, MPI_Comm comm)
```

sendbuf: send buffer

sendcount: number of elements to send

sendtype: type of data

recvbuf: receive buffer

recvcount: number of elements to receive

recvtype: type of data

comm: communicator

Same as `MPI_Gather` but without the root

Reduction collectives

- In OpenMP there are reduction variables
- MPI has reduction functions which perform a similar role
- **MPI_Reduce**: carry out reduction and return result to the specified process
- **MPI_Allreduce**: carry out reduction and return result to all processes
- Communication overhead is greater if we return result to all processes

MPI_Reduce

```
int MPI_Reduce(const void *sendbuf, void  
*recvbuf, int count, MPI_Datatype type,  
MPI_Op op, int root, MPI_Comm comm)
```

sendbuf: send buffer

recvbuf: receive buffer

count: number of elements to send/receive

type: type of data

op: reduction operator

root: rank of root process

comm: communicator

Reduction operators

MPI_MAX	maximum	integer, float
MPI_MIN	minimum	integer, float
MPI_SUM	sum	integer, float, complex
MPI_PROD	product	integer, float, complex
MPI_LAND	logical and	C integer, logical
MPI_BAND	bit-wise and	C integer, logical, byte
MPI_LOR	logical or	C integer, logical
MPI_BOR	bit-wise or	C integer, logical, byte
MPI_LXOR	logical xor	C integer, logical
MPI_BXOR	bit-wise xor	C integer, logical, byte

Other reduction operators are available

MPI_Allreduce

```
int MPI_Allreduce(const void *sendbuf,  
void *recvbuf, int count, MPI_Datatype  
type, MPI_Op op, MPI_Comm comm)
```

sendbuf: send buffer

recvbuf: receive buffer

count: number of elements to send and receive

type: type of data

op: reduction operator

comm: communicator

Same as **MPI_Reduce** but without the root

Unit Summary

- MPI has collective communications which take place between the group of processes defined by a communicator
- Examples of MPI collectives:
 - `MPI_Bcast`
 - `MPI_Scatter`
 - `MPI_Gather/MPI_Allgather`
 - `MPI_Reduce/MPI_Allreduce`
- Collective communication patterns can be one-to-all, all-to-one or all-to-all